1st lab doesn't have any evaluation.
Evaluation starts from 2nd lab.

(lab-2) 1st Lab Evaluation

Question:

Using the Vandermonde matrix method, write a Python program to construct the interpolating polynomial that passes through the following data points:

x	-0.79	-0.65	0.56	1.50
y	1.42	1.73	1.93	2.74

Then use the obtained polynomial to estimate the values of the function at **x=-0.5 and 1.**

Sol:

```
def get_poly(data_x, data_y):
    n_nodes = len(data_x)

    X = np.zeros((n_nodes, n_nodes))




    for i in range(n_nodes):
      for j in range(n_nodes):
       X[i][j]=data_x[i]**j
    print(X)

    X_inv = np.linalg.pinv(X)
    a = np.dot(X_inv, data_y)
    p = Polynomial(a)
```

```
    return p
```

```
data_x = np.array([-0.65, 0.56,1.5,-0.79])
data_y = np.array([1.73,1.93,2.74, 1.42])
p = get_poly(data_x, data_y)
X=[-0.5,1]
```

##
Suppose, you have three nodes (-0.75, 1.87), (0.5, 2.20), (1.5, 2.44). Using Vandermonde Matrix method, print out the value of the interpolating polynomial at x = 3. You have to solve the given problem using above implemented get_poly() method.

##
You have been given three separate set of nodes,

Set 1: (-0.45, 1.02), (0.39, 1.47), (1.33, 2.02)

Set 2: (0.5, 1.24), (-0.39, -1.46)

(a)Find two separate interpolating polynomial equations using the given set of nodes.

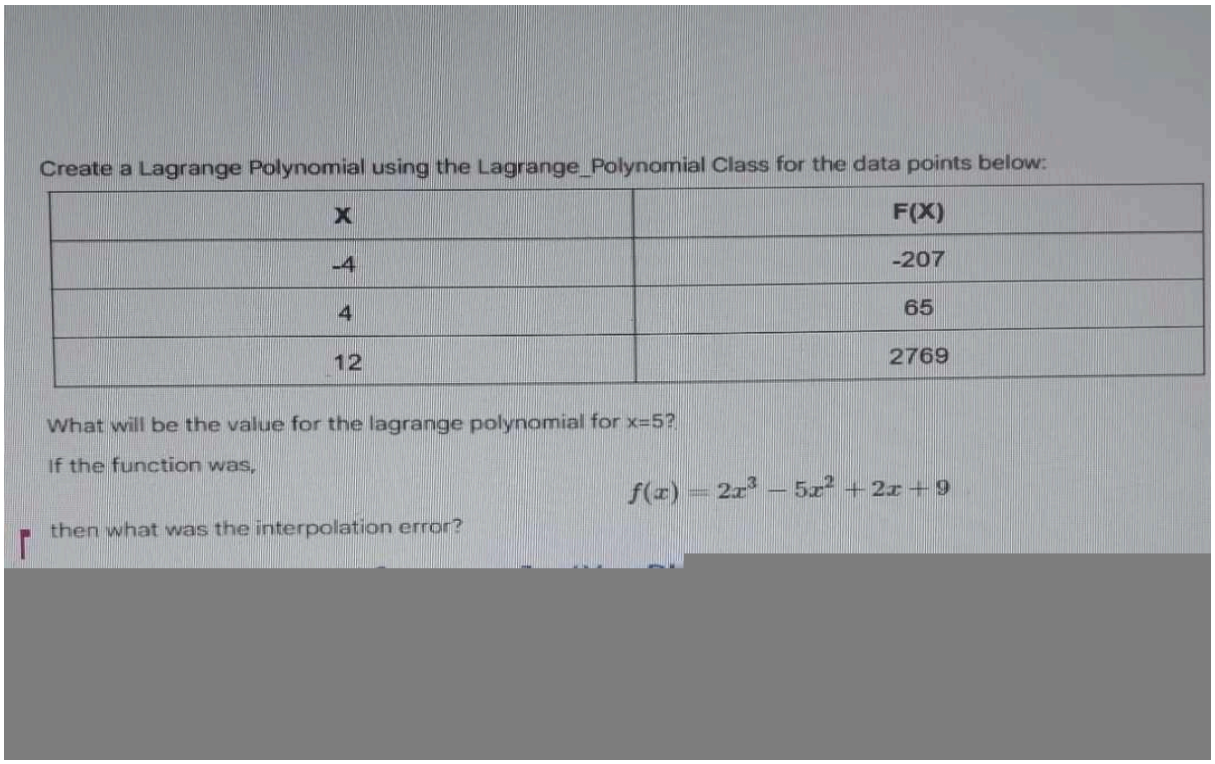
(b)Print the degrees and the coefficients of each of the polynomials separately.

(c)Calculate and print the absolute average value of the coefficients for each of the polynomials separately.

(d)Finally use the given values of x to find their corresponding y values for the polynomial with the highest average of coefficients. [Hint: You can take decision based on the average value of the coefficients]

Given x value list = [-0.45, 0.51, 1.23, 1.49, 1.67, 2.05, 2.77]

(lab-3) 2nd lab evaluation question:



Create a Lagrange Polynomial using the Lagrange_Polynomial Class for the data points below:

X	F(X)
-4	-207
4	65
12	2769

What will be the value for the lagrange polynomial for x=5?

If the function was,

$$f(x) = 2x^3 - 5x^2 + 2x + 9$$

then what was the interpolation error?

##

Using the Lagrange interpolation method, write a Python program to construct the interpolating polynomial that passes through the following data points:

x	-4	4	12
y	-207	65	2770

Evaluate the interpolating polynomial at 5, 7 and 9. The actual function is

$$f(x) = 2x^3 - 5x^2 + 2x + 9.$$

Compute the interpolation error at each of these points by comparing the interpolated values with the exact values of f(x).

Sol:

```
data_x = np.array([-4,4,12])
data_y = np.array([-207,65,2770])
p = Lagrange_Polynomial(data_x, data_y)
x_arr=np.array([5,7,9])
p_x_arr=p(x_arr)
f_x=lambda x: 2*x**3-5*x**2+2*x+9
f_x_arr=f_x(x_arr)

error = (p_x_arr-f_x_arr)

print("Interpolation error: ")
print(error)
```

##

Using the **Lagrange interpolation method**, write a Python program to construct the interpolating polynomial that passes through the following data points:

x	y
−2	−8
0	2
3	15
5	40

Evaluate the interpolating polynomial at the points

$$x = -1,\ 1,\ 2,\ 4.$$

The actual function is

$$f(x) = x^2 + 2x + 2.$$

Compute the **absolute interpolation error** at each of the given points and determine the **maximum interpolation error.**

Sol:

```
data_x = np.array([-2,0,3,5])
data_y = np.array([-8,2,15,40])
x_arr=[-1,1,2,4]

p = Lagrange_Polynomial(data_x, data_y)
print(p)

error_arr=[]
lagrange=Lagrange_Polynomial(data_x, data_y)
f_x_arr=[]

for i in x_arr:
    f_x=i**2+2*i+2
    f_x_arr.append(f_x)

print(f_x_arr)
p_x_arr=p(x_arr)
print(p_x_arr)

for j in range(len(f_x_arr)):
  error=abs(p_x_arr[j]-f_x_arr[j])
  error_arr.append(error)

print(error_arr)
print(max(error_arr))
```

(lab-4) 3rd lab evaluation question:

x	f(x)	f'(x)
1.5	2.45	-1
2	5	2
4.33	7	3.4

Using **Hermite interpolation**, construct the interpolating polynomial for the above given data. And Evaluate the polynomial at x=1.56

Sol:

```
x = np.array([1.5,2,4.33])
y = np.array([2.45,5,7])
y_prime =  np.array([-1,2,3.4])
f_interp = hermit(x, y, y_prime)

x_eval = 1.56
y_eval = f_interp(x_eval)
print(f'Hermite:{f_interp}')

print(f"{y_eval}")
```

Task 01:

Suppose, consider the following data set:

x	$f(x)$	$f'(x)$
0	2	1
1	4	-1
3	5	-2

Using Hermit basis, print out the interpolating polynomial and find the value at x = [0.55, 2.75, 3.15].

You have to solve this problem using the hermit function.

##

Using the **Newton's Divided Difference interpolation method**, write a Python program to construct the interpolating polynomial that passes through the following data points:

x	y
−1.50	3.75
0.50	1.22
1.67	2.90

First, compute the **divided difference coefficients**, then construct the Newton interpolating polynomial. Finally, use the polynomial to estimate the value of the function at

$$x = 0.25.$$

Sol:

```
data_x=[-1.5,0.5,1.67]
data_y=[3.75,1.22,2.9]
```

```
differences = calc_div_diff(data_x, data_y)
p = Newtons_Divided_Differences(list(differences),data_x)
test_x=np.array([0.25])
test_y = p(test_x)
```

(lab-5) 4th lab evaluation question:

Using Richardson extrapolation, approximate the first derivative of the polynomial

$$f(x) = 2 + x - 6x^2 - 2x^3 + 2.5x^4 + x^5$$

at the points

$$x = 0 \quad \text{and} \quad x = -1.18625,$$

using a step size of $h = 0.1$.

Sol:

```
f= Polynomial ([2, 1, -6, -2, 2.5, 1])
x1= 0
h=0.1
ddh1= dh1(f, h, x1)
print(ddh1)
x2= -1.18625
```

```
ddh2= dh1(f, h, x2)
print(ddh2)
```

##

Using Richardson extrapolation, approximate the first derivative of

$$f(x) = 5x^4 - 3x^3 + 6x^2 - 4$$

at $x = 1$ with step size $h = 0.5$.

Sol:

```
def f(x):
  return 5*x**4-3*x**3+6*x**2-4
x_val=1
h=0.5
result=dh1(f, h, x_val)
print(result
```

##

Task 02:

Consider the following function:

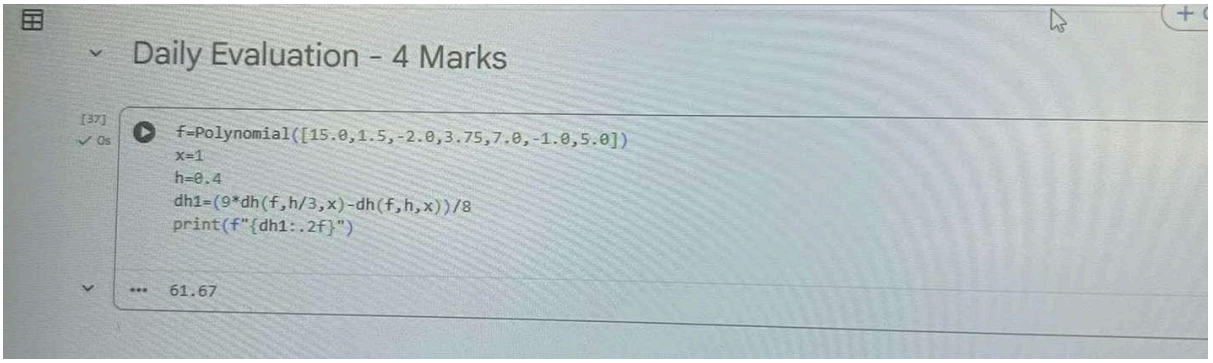
$$f(x) = 5x^6 - x^5 + 7x^4 + 3.75x^3 - 2x^2 + 1.5x + 15$$

Now, compute the First Order Richarson Extrapolation, `D^(1)` at `x = 1` and `h = 0.4` when the formula is derived using `h -> h/3`. Print the answer upto 2 decimal places.

N.B. You can use the previously learned methods while solving the problem.

$$D_h^{(1)} = \frac{9D_{h/3} - D_h}{8}$$

Sol:



##

Task 1

Let,

$$f(x) = x^5 + 2.5x^4 - 2x^3 - 6x^2 + x + 2$$

a. Using Richardson's Extrapolation, what is the slope of f(x) at x=0, −1.18625 and step size = 0.1?

b. Compare the error of your method with actual, forward, backward and central differentiation at x=0, −1.18625

c. Plot actual derivative, Forward derivative, Backward derivative, Central derivative and the derivative using Richardson's Extrapolation in a graph. Here,

$h=0.1, \quad -2 \leq x \leq 1.2$

Sol:

```
f=Polynomial([1.0,2.5,-2.0,-6.0,1.0,2.0])
x=np.array([0,-1.18625])
#a
diff1=dh1(f, 0.1, x)
print(diff1)

#b
for i in range(len(x)):
  f_prime = f.deriv(1)
  Y_actual = f_prime(x[i])
  my_error=Y_actual-dh1(f, 0.1, x[i])
  print("actual-Richad",Y_actual-diff1[i])
```

```
  forward=forward_diff(f, 0.1, x[i])
  print("actual-forward",Y_actual-forward)
  backward=backward_diff(f, 0.1, x[i])
  print("actual-backward",Y_actual-backward)
  central=central_diff(f, 0.1, x[i])
  print("actual-central",Y_actual-central)


backward=backward_diff(f, h, x)




fig, ax = plt.subplots()
ax.axhline(y=0, color='k')
p = Polynomial([1.0,2.5,-2.0,-6.0,1.0,2.0])
data = p.linspace(domain=[-2, 1.2])
ax.plot(data[0], data[1], label='Function')

p_prime = p.deriv(1)
data2 = p_prime.linspace(domain=[-2, 1.2])
ax.plot(data2[0], data2[1], label='Derivative')

ax.legend()



h = 0.1
fig, bx = plt.subplots()
```

```
bx.axhline(y=0, color='k')

y = forward_diff(p, h, x)
bx.plot(x, y, label='Forward; h=0.1')
y = backward_diff(p, h, x)
bx.plot(x, y, label='Backward; h=0.1')
y = central_diff(p, h, x)
bx.plot(x, y, label='Central; h=0.1')

data2 = p_prime.linspace(domain=[-2, 1.2])
bx.plot(data2[0], data2[1], label='actual')

bx.legend()




fig, ax = plt.subplots()
ax.axhline(y=0, color='k')

draw_graph(p_prime, ax, [-2,1.2], 'actual')

h = 0.1
y = dh1(p, h, x)
ax.plot(x, y, label='Richardson; h=0.1')

ax.legend()
```

Using the Polynomial class, write a Python program to construct the polynomial

$$f(x) = -\frac{1}{13}x^3 + 2x^2 - 9.5x - 10.$$

Find all the roots (zeros) of the polynomial, evaluate the polynomial at the obtained roots to verify the results, and plot the points $(x,, f(x))$ corresponding to the roots on a graph.

Sol:

```
 f=Polynomial([-10,-9.5,2,(-1/13)])
roots=f.roots()
print(roots)
xs=roots
ys=f(xs)
plt.plot(xs,ys,"go")
plt.plot(xs,ys)
```

Using the **Bisection Method**, write a Python program to find a root of the polynomial

$$f(x) = -\tfrac{1}{13}x^3 + 2x^2 - 9.5x - 10.$$

Use the initial interval

$$[a, b] = [-10,\ 0]$$

and a stopping criterion of

$$\varepsilon = 10^{-4}.$$

Sol:

```
 f=Polynomial([-10,-9.5,2,(-1/13)])
a = 0
b = -10
m = (a + b) / 2
e = 1e-4

root = 0.0

list_a = []
list_b = []
list_m = []
list_f = []
```

```
root_found=False
while root_found==False:
   list_a.append(a)
   list_b.append(b)
   list_m.append(m)
   list_f.append(f(m))

   if f(a)*f(m)<0:
      b=m
   elif f(a)*f(m)>0:
      a=m
   elif f(a)*f(m)==0:
    root=m
    root_found=True
    break
   old_m=m
  m=(a+b)/2

    if (abs(m-old_m)/abs(m)<=e):
      root=old_m
      root_found=True


print(root)
```

(lab-6) 5th lab evaluation question

> Write a program to implement the **Bisection Method** for solving
>
> $$x^4 + x^2 - 25x + 5 = 0,$$
>
> using:
>
> - **"Initial interval:** $[1, 3]$**"**
> - **"Tolerance:** 10^{-4}**"**
>
> Display the iteration table containing a, b, midpoint m, and $f(m)$, and determine the approximate root.

Sol:

```
f=Polynomial([5,-25,1,0,1])
a=1
b=3
e=1e-4
m = (a + b) / 2
list_a = []
list_b = []
list_m = []
list_f = []
root_found =False
m = (a + b) / 2
val = float('inf')

while val >= e:
    list_a.append(a)
    list_b.append(b)
    list_m.append(m)
    list_f.append(f(m))

    if f(a) * f(m) < 0:
        b = m
    else:
        a = m

    new_m = (a + b) / 2
    val = abs(new_m - m)
    root=m
    m = new_m

print(root)
```

##

Using the **Bisection Method**, write a Python program to find a root of the polynomial

$$f(x) = -\tfrac{1}{13}x^3 + 2x^2 - 9.5x - 10.$$

Use the initial interval

$$[-10,\ 0]$$

and a tolerance of

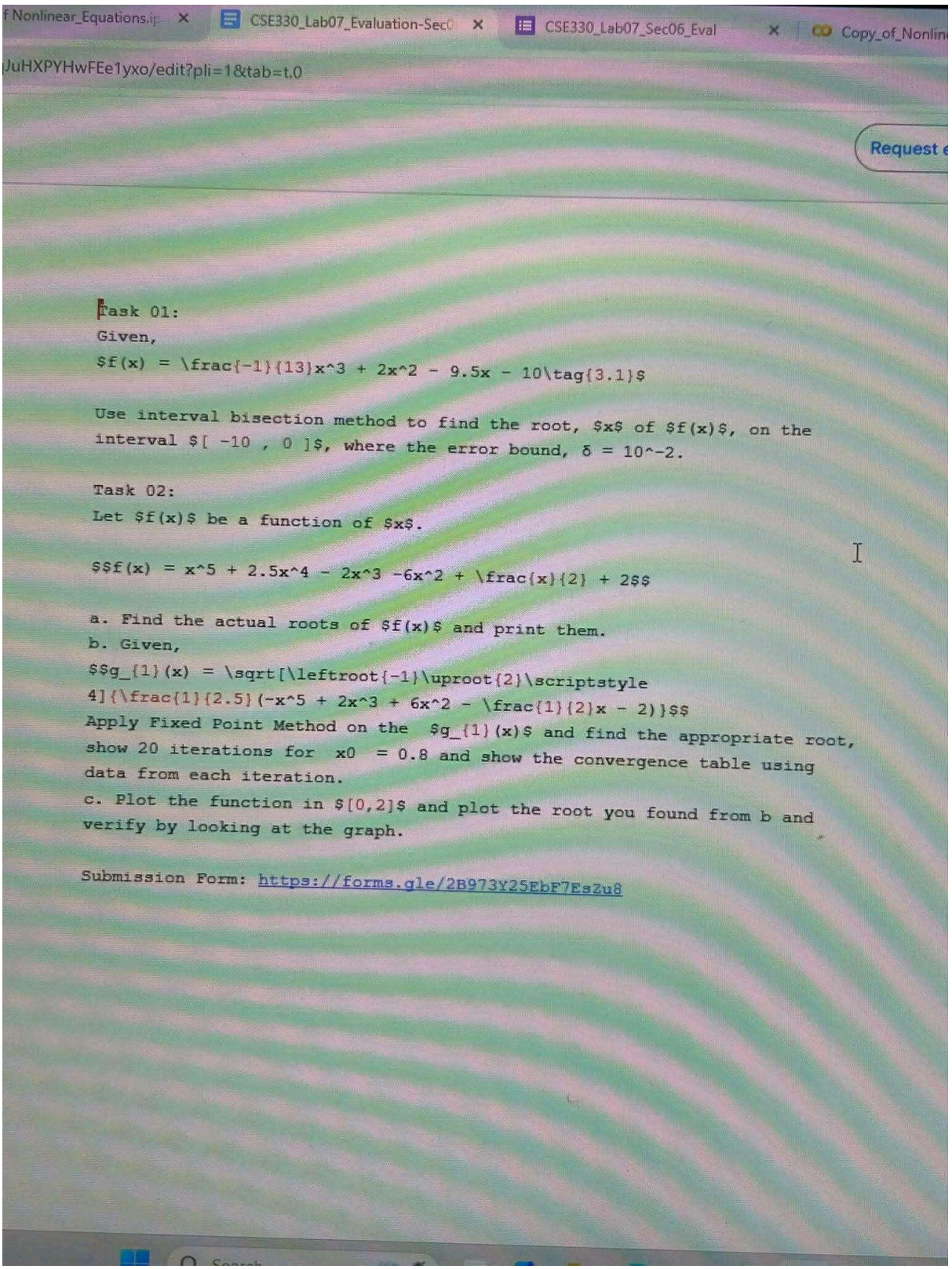
$$10^{-4}.$$

Sol:

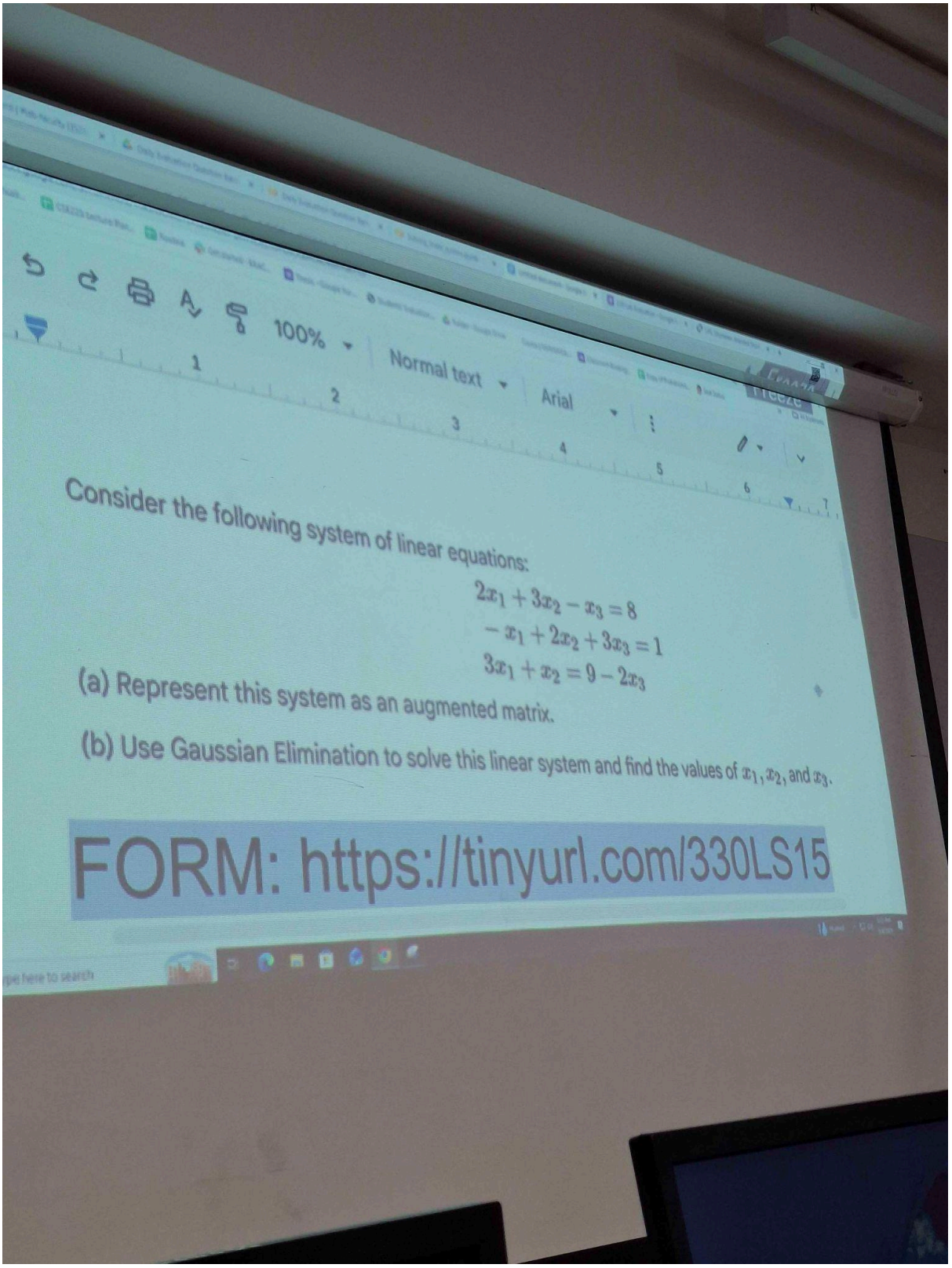
```
f=Polynomial([-10,-9.5,2,(-1/13)])
a = 0
b = -10
m = (a + b) / 2
e = 1e-4

root = 0.0
```

```
list_a = []
list_b = []
list_m = []
list_f = []


root_found=False
while root_found==False:
    list_a.append(a)
    list_b.append(b)
    list_m.append(m)
    list_f.append(f(m))

    if f(a)*f(m)<0:
        b=m
    elif f(a)*f(m)>0:
        a=m
    elif f(a)*f(m)==0:
      root=m
      root_found=True
      break
    old_m=m
    m=(a+b)/2

    if (abs(m-old_m)/abs(m)<=e):
      root=old_m
      root_found=True
```

print(root)


f Nonlinear_Equations.ip
CSE330_Lab07_Evaluation-Sec0
CSE330_Lab07_Sec06_Eval
Copy_of_Nonlin
JuHXPYHwFEe1yxo/edit?pli=1&tab=t.0
Request
Task 01:
Given,
$f(x) = \frac{-1}{13}x^3 + 2x^2 - 9.5x - 10\tag{3.1}$
Use interval bisection method to find the root, x of $f(x)$, on the interval $[-10 , 0]$, where the error bound, δ = 10^-2.
Task 02:
Let $f(x)$ be a function of x.
$$f(x) = x^5 + 2.5x^4 - 2x^3 -6x^2 + \frac{x}{2} + 2$$
a. Find the actual roots of $f(x)$ and print them.
b. Given,
$$g_{1}(x) = \sqrt[\leftroot{-1}\uproot{2}\scriptstyle 4]{\frac{1}{2.5}(-x^5 + 2x^3 + 6x^2 - \frac{1}{2}x - 2)}$$
Apply Fixed Point Method on the $g_{1}(x)$ and find the appropriate root, show 20 iterations for x0 = 0.8 and show the convergence table using data from each iteration.
c. Plot the function in $[0,2]$ and plot the root you found from b and verify by looking at the graph.
Submission Form: https://forms.gle/2B973Y25EbF7EsZu8

(lab-8) 7th lab evaluation question:


100%
Normal text
Arial
Consider the following system of linear equations:
$2x_1 + 3x_2 - x_3 = 8$
$-x_1 + 2x_2 + 3x_3 = 1$
$3x_1 + x_2 = 9 - 2x_3$
(a) Represent this system as an augmented matrix.
(b) Use Gaussian Elimination to solve this linear system and find the values of x_1, x_2, and x_3.
FORM: https://tinyurl.com/330LS15

#######

ng_linear_system.ipynb - C | Copy of Solving_linear_system... | CSE330-06 Lab 8 Evaluation | CSE330 Lab8 Eval

/C1YHFNnxr7o_yc/edit?tab=t.0

Google Chrome isn't your default browser **Set as default**

Request edit a

Question 1:

Given the following **2x3 augmented matrix A:**

$$\left[\begin{array}{cc|c} 3 & 6 & -9 \\ 1 & 3 & 4 \end{array}\right]$$

Write a function that performs a single step of Gaussian elimination to modify the **second row (row 1)** using the **first row (row 0)**. Specifically, the function should:

1. Calculate the multiplier needed to make the first element of the second row zero.
2. Subtract the necessary multiple of the first row from the second row.

Question 2:

Consider the following 4x4 matrix and determine whether Gaussian elimination can be used here. Create the matrix and show necessary working for your conclusion.

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 8 \\ 3 & 6 & 9 & 12 \\ 4 & 8 & 12 & 16 \end{bmatrix}$$

Search

ASUS

Solution:

colab.research.google.com/drive/1S8Z5HX1a_o75IJAdBtf66D1pJmRp7qpe#scro

Copy of Solving_linear_system.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[33]
```
results = [11, -2.5, -6]

np.testing.assert_array_equal(test, results)
```

Daily Evaluation - 4 Marks

[41]
✓ 0s
```
data_n = 2
data_A = np.array([[3, 6, -9], [1, 3, 4]])
test = get_result_gaussian_elimination(data_n, data_A)
print(data_A)
print("test result")
print(test)
```

```
[[ 3  6 -9]
 [ 1  3  4]]
test result
[-17.   7.]
```

[42]
✓ 0s
```
data_A = np.array([[1, 2, 3, 4], [2, 4, 6, 8], [3, 6, 9, 12]])
print(data_A)

data_n=3
test = get_result_gaussian_elimination(data_n, data_A)
print(test)
```

```
[[ 1  2  3  4]
 [ 2  4  6  8]
 [ 3  6  9 12]]
found diagonal has 0
None
```

Variables Terminal

78°F
Mostly cloudy

##

Using Python and NumPy, write a program to compute the inverse of the matrix

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}.$$

Sol:

```
A = np.array([[2,-1,0], [-1,2,-1], [0,-1,2]])
inverse=np.linalg.inv(A)
I=np.dot(A,inverse)
print(abs(np.round(I)))
```

##

Using Python and NumPy, write a program to compute the determinant of the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 8 \\ 3 & 6 & 9 & 12 \\ 4 & 8 & 12 & 16 \end{bmatrix}.$$

Determine whether the matrix is **singular** or **non-singular** based on the value of its determinant.

Sol:

```
A=[[1,2,3,4],[2,4,6,8],[3,6,9,12],[4,8,12,16]]
determinant=np.linalg.det(A)
print(determinant)
```